
Федеральное государственное автономное образовательное учреждение
высшего образования

«Московский физико-технический институт
(национальный исследовательский университет)»
Физтех-школа Прикладной Математики и Информатики
Кафедра банковских информационных технологий

Направление подготовки / специальность: 03.03.01 Прикладные математика и физика

Направленность (профиль) подготовки: Математическая физика, компьютерные технологии и
математическое моделирование в экономике

СРАВНЕНИЕ ТРАНСФОРМЕРА И ГРАДИЕНТНОГО БУСТИНГА В ЗАДАЧЕ КРЕДИТНОГО СКОРИНГА

(бакалаврская работа)

Студент:

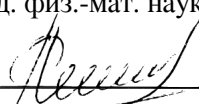
Царев Иван Николаевич



(подпись студента)

Научный руководитель:

Филипенков Николай Владимирович,
канд. физ.-мат. наук



(подпись научного руководителя)

Консультант (при наличии):

(подпись консультанта)

Москва 2025

Аннотация

В рамках данного исследования рассматриваются современные подходы к решению задачи кредитного скоринга. Цель данной работы - сравнение передовых архитектур нейронных сетей, преимущественно основанных на применении трансформерных слоев, и методов классического машинного обучения, а именно, градиентного бустинга в задаче кредитного скоринга.

В ходе работы были спроектированы и реализованы несколько архитектур нейронных сетей, адаптированных для работы с табличными данными. Кроме того, реализовано несколько методов предварительной обработки данных - эмбедингов, которые оказывают существенное влияние на качество модели. Все полученные модели были протестированы на нескольких публичных датасетах и сравнены с реализацией градиентного бустинга из библиотеки `CatBoost`, которая в настоящее время считается одной из самых эффективных.

Основным результатом работы является демонстрация того, что нейронные сети способны эффективно работать в задаче кредитного скоринга и обеспечивать качество предсказаний, сопоставимое с современными реализациями градиентного бустинга. Более того, при правильной настройке нейронных сетей возможно достижение более высоких результатов. Представленный подход показывает потенциал использования нейронных сетей даже в таких специфических задачах, как кредитный скоринг, и может стимулировать дальнейшие исследования в области применения глубокого обучения к табличным данным.

Содержание

Содержание	3
1 Введение	5
1.1 Кредитный скоринг	5
1.2 История решения задачи кредитного скоринга	6
1.3 Методы решения	7
1.4 Постановка задачи и цели работы	9
2 Архитектура нейронной сети	10
2.1 Baseline	10
2.1.1 MLP	10
2.1.2 ResNet	11
2.2 Трансформеры	12
2.2.1 Что такое трансформер?	12
2.2.2 Трансформер в кредитном скоринге - TabTransformer	13
2.2.3 FT-Transformer	14
2.2.4 MLP encoder Transformer	15
2.3 Примеры эмбедингов	16
2.3.1 Element-wise multiplication encoding	17
2.3.2 Piecewise linear encoding	17
2.3.3 Periodic encoding	19
3 Данные и их обработка	20
3.1 Данные	20
3.2 Обработка данных	20
4 Эксперименты	22
4.1 Метрики качества модели	22
4.1.1 Precision и Recall	22
4.1.2 ROC-AUC	22
4.1.3 Выводы о метриках	23
4.2 Эксперименты	24

4.2.1 Структура репозитория	24
4.2.2 Сравнение бейзлайнов моделей	24
4.2.3 Влияние эмбедингов численных признаков на качество модели .	25
4.2.4 Финальный эксперимент	26
5 Заключение	28
Список литературы	29

1 Введение

1.1 Кредитный скоринг

Кредитный скоринг играет ключевую роль в современном финансовом мире. Скоринг - процесс, результатом которого, является оценка кредитоспособность клиента (обычно на выходе ожидается вероятность дефолта клиента). Современные скоринговые модели учитывают множество поведенческих признаков пользователя, таких как кредитная история, финансовое положение, личные данные и даже история его покупок на маркетплейсах, поездок на такси и заказов в сервисах доставки еды. Важно понимать, что кредитный скоринг помогает банкам минимизировать риски, связанные с невозвратом кредитов, обеспечивая их более эффективную выдачу.

Актуальность задачи кредитного скоринга в наши дни сложно переоценить. В условиях быстро меняющейся экономической ситуации точные модели кредитного скоринга становятся жизненно необходимыми для крупных компаний. Скоринг используется повсюду: в банках, микрофинансовых организациях (МФО), BNPL-сервисах (*Buy Now, Pay Later*) и страховых фирмах.

Для бизнеса кредитный скоринг – это способ снижения финансовых рисков и оптимизации внутренних процессов. Благодаря автоматизации этого процесса и внедрению современных технологий машинного обучения, компании могут сократить время обработки заявок до нескольких миллисекунд.

Успешное управление кредитными рисками способствует увеличению прибыли и укреплению финансовых показателей. Например, в крупнейших банках России процентные доходы составляют значительную долю от прибыли: Т-Банк¹ в своем отчете за 2023 год показал, что 37% от всей прибыли банка это процентные доходы.

С другой стороны, качественный кредитный скоринг влияет на повседневную жизнь граждан, обеспечивая им более легкий доступ к финансовым ресурсам. Люди могут быстрее получать кредиты на крупные покупки, такие как техника, недвижимость или автомобиль.

¹Источник: журнал Тинькофф, 2023

1.2 История решения задачи кредитного скоринга

Кредитный скоринг прошел долгий путь развития от ручных правил до современных сложных и автоматизированных систем.

В первых скоринговых системах решения принимались кредитными экспертами вручную, основываясь на их опыте и субъективном мнении. Этот процесс был не только трудоемким, но и подверженным человеческим факторам, что ограничивало его масштабируемость, точность и непредвзятость.

Повышение спроса на кредитные услуги, особенно с введением кредитных карт, подчеркнуло необходимость автоматизации принятия решений. В 1941 году Давид Дюрэн представил первую исследовательскую работу по кредитному скорингу, где он оценивал значимость различных факторов. Это положило начало развитию более структурированной методологии в этой области.

Ключевой вехой стало появление в 1956 году компании FICO, которая начала активно разрабатывать автоматизированные системы для оценки кредитоспособности. В 1960-х годах началось внедрение компьютерных технологий для обработки больших объемов данных, что существенно повысило эффективность скоринга.

Логистическая регрессия сделала возможной более точную и объективную оценку, позволяя обрабатывать множество параметров в автоматическом режиме. Однако на этом технологический прогресс не остановился: в начале 2000-ых появились более сложные методы, такие как ансамбли деревьев решений, в частности методы градиентного бустинга.

Сейчас же на пороге стоят нейронные сети, которые превосходят классические ML-алгоритмы в эффективности во многих сферах. Задача кредитного скоринга приносит огромную прибыль крупнейшим корпорациям мира, делая ее решение передовой научной областью и предметом конкурентной борьбы за точность.

1.3 Методы решения

За последние два десятилетия градиентный бустинг стал наверное самым популярным алгоритмом машинного обучения для задач на табличных данных. Он применяется во всех BigTech компаниях для большого числа задач, таких как ранжирование поисковой выдачи, прогнозирование спроса, предсказание оттока пользователей и, конечно, для кредитного скоринга. Популярность градиентного бустинга на деревьях, связана с его высокой эффективностью, простотой и доступностью (есть множество библиотек, в которых GBDT уже реализован).

Нейронные сети, напротив, зарекомендовали себя в областях, требующих обработки сложных и неструктурированных данных: изображений, текстов, звуков и последовательностей. Способность выявлять глубокие нелинейные зависимости сделала их незаменимыми в задачах компьютерного зрения, обработки естественного языка и распознавания речи. В рамках кредитного скоринга также существуют задачи, которые уже давно и успешно решаются нейросетями. Например, многие банки используют различные виды рекуррентных нейронов (RNN) для анализа клиента по истории его транзакций.

Однако при работе с табличными данными - данными хранящимися в виде таблицы, где каждая строка - отдельный объект, а каждый столбец — признак объекта, нейронные сети до сих пор не получили широкого распространения [12]. В этих случаях градиентный бустинг продолжает оставаться самым популярным инструментом благодаря своей эффективности и оптимальному соотношению затрат на обучение и полученного качества.

В последние несколько лет интерес к применению глубокого обучения на табличных данных заметно возрос. Исследователи из таких компаний, как *Yandex*, *Google*, *Amazon* каждый год представляют на конференциях уровня NIPS новые статьи посвященные этой задаче. А работы [1, 2, 5, 6] показывают, что с помощью специальных архитектур нейронных сетей, взятых зачастую совсем из другой задачи (например: из обработки естественного языка или компьютерного зрения), адаптированных к формату табличных данных, можно получить результаты, сопоставимые классическим методам.

Несмотря на потенциал, нейронные сети сталкиваются с рядом сложностей при

применении к табличным данным. Их разработка требует тщательного продумывания и написания архитектуры, а обучение - длительное и ресурсоемкое. Также, для нейронок может быть критически важным наличие большого объема данных для эффективного обучения, что особенно проблематично для задачи кредитного скоринга, ведь каждый таргет в обучении - потерянные деньги банка.

Кроме того, выданные кредиты уже проходят какую-то проверку (модельную или ручную), а значит все данные в обучении - смещенные, они характеризуют только часть потока, что может плохо сказаться на разделяющей способности модели. Чтобы избежать такой проблемы, банки обычно запускают шлюз (часть потока, на которой не применяются никакие рисковые правила), чтобы исследовать весь поток целиком. При этом, шлюз обычно очень небольшого размера из-за высокой стоимости.

Несмотря на это нейронки обладают важными преимуществами над методами классического машинного обучения:

- Экономия времени на обработку датасетов: разработчикам не нужно придумывать и генерировать фичи вручную, нейронка работает как черный ящик, на вход которому достаточно подать всю информацию.
- Упрощенная работа с разнородными признаками: нейронки хорошо работают как с временными рядами (например транзакциями), так и с обычными табличными данными. Бустинг же с последовательными данными работать в принципе не умеет, поэтому их нужно вручную преобразовывать в фичи (например путем агрегации по окнам разной длины).
- Быстрая адаптация к быстро меняющимся рыночным условиям путем постоянного дообучения существующей модели на новых данных.

Новизна темы диплома заключается в исследовании применения различных адаптаций современных нейронных сетей для задачи кредитного скоринга на табличных данных, что сейчас является недоизученным. Традиционные методы, такие как градиентный бустинг, доминируют в этой области, поэтому потенциальное использование нейронных сетей может открыть новые возможности. Это перспективное направление объединяет передовые технологии в сфере глубинного обучения с практическими задачами, представляющими высокий интерес для крупного бизнеса.

1.4 Постановка задачи и цели работы

Целью данной работы является проведение сравнительного анализа различных архитектур нейронных сетей и алгоритмов градиентного бустинга на примере задачи бинарной классификации на табличных данных, а именно - задачи кредитного скоринга.

Основная задача - реализовать адаптированные архитектуры нейронных сетей и провести их оценку на различных банковских датасетах с пользовательскими фичами. Для этого используются открытые данные, такие как датасеты с соревнований на платформе **Kaggle**, организованных различными крупными банками, например *HomeCreditBank* и *Raiffeisen*.

Для обеспечения объективности и значимости результаты нейронных сетей сравниваются с показателями популярных алгоритмов градиентного бустинга, а именно с **CatBoost**, который в настоящее время считается одним из лучших алгоритмов для обработки табличных данных и широко применяется в индустрии благодаря своей высокой точности, скорости и простоте использования.

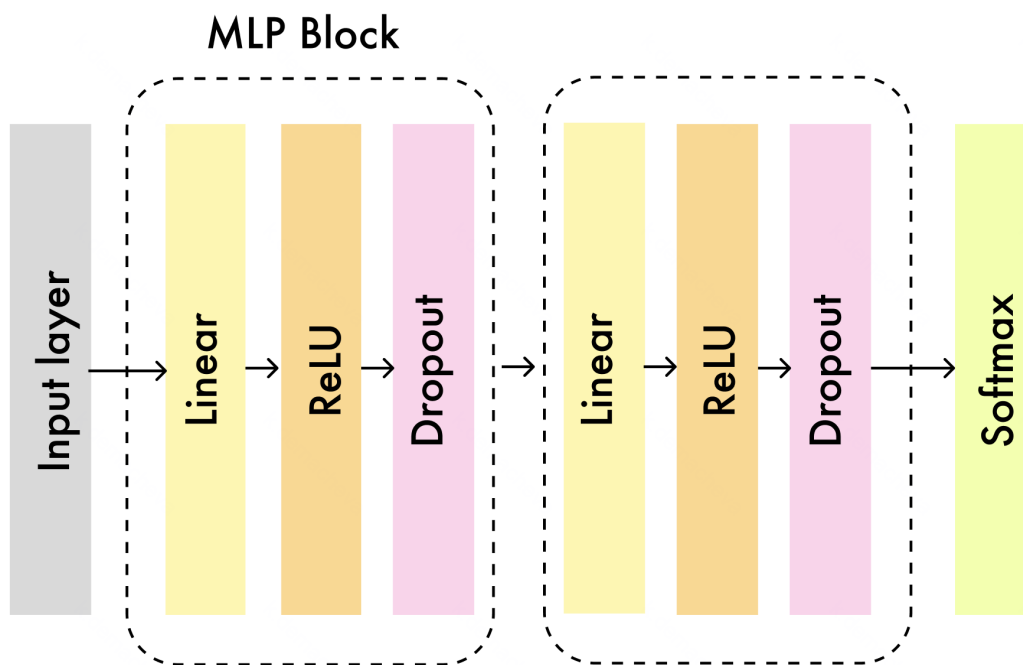
2 Архитектура нейронной сети

2.1 Baseline

Для оценки эффективности продвинутых архитектур нейронных сетей необходимы базовые модели, с которыми можно сравнивать результаты. В качестве таких бейзлайнов, я выбрал две очень простые, но эффективные архитектуры - MLP и ResNet.

2.1.1 MLP

Полносвязная нейронная сеть или многослойный перцептрон (MLP) - самая простая архитектура искусственной нейронной сети, в которой каждый нейрон соединен со всеми нейронами следующего слоя.



MLP состоит из блоков, в каждом из которых выполняются следующие операции:

1. Линейное преобразование над входящими данными $x \in \mathbb{R}^d$, с обучаемыми параметрами: матрица весов $W \in \mathbb{R}^{d \times k}$ и вектор смещения $b \in \mathbb{R}^k$

$$x \mapsto xW + b, \quad (1)$$

2. Применение функции активации - нелинейного преобразования, поэлементно

применяющееся к пришедшим на вход данным:

$$x \mapsto \sigma(x) = \sigma(xW + b) \quad (2)$$

где $\sigma(\cdot)$ — какая-то нелинейная функция, например:

$$\text{ReLU: } \sigma(x) = \max(0, x)$$

$$\text{Sigmoid: } \sigma(x) = \frac{1}{1 + \exp^{-x}}$$

Кроме того, в архитектуре MLP часто применяется функция активации SELU, которая была предложена в статье "*Self-Normalizing Neural Networks*" (2017) [8].

$$\text{SELU: } \sigma(x) = \lambda \begin{cases} x, & \text{если } x > 0 \\ \alpha e^x - \alpha, & \text{если } x \leq 0 \end{cases} \quad (3)$$

Автор утверждает, что SELU (Scaled Exponential Linear Unit) решает проблемы затухания и взрыва градиентов, ускоряя обучение MLP.

3. Dropout - метод регуляризации нейронных сетей, который на этапе обучения случайно отключает (зануляет выходы) некоторые нейроны с вероятностью p .

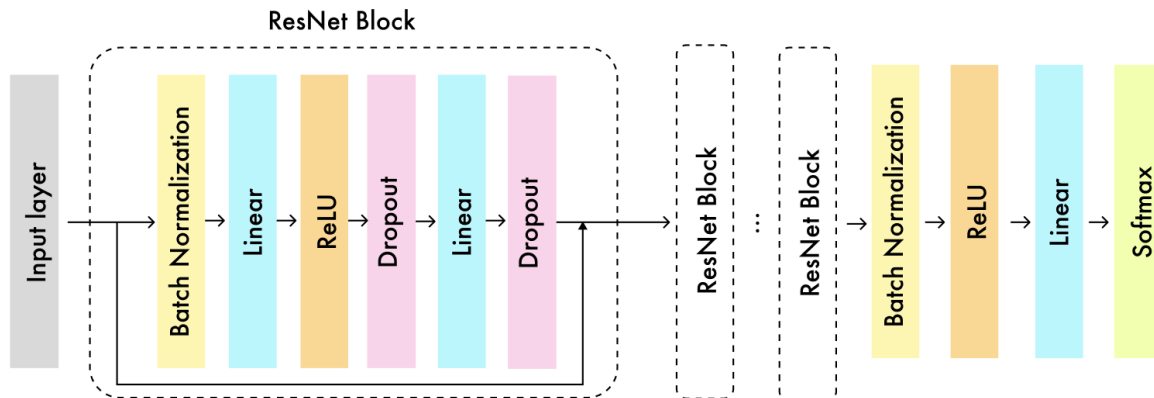
Модель MLP требует относительно небольших вычислительных ресурсов в процессе обучения, поэтому в дальнейшем именно на этой архитектуре проводилось тестирование различных типов эмбедингов.

2.1.2 ResNet

Нейронные сети достигли значительных успехов в различных областях, однако по мере увеличения глубины сети возникает проблема затухающих градиентов, что затрудняет обучение глубоких моделей. Поэтому в 2015 году команда под руководством Кайминга Хэ предложила архитектуру Residual Network (ResNet) [10], которая позволила эффективно обучать сети глубиной в сотни слоёв.

Ключевая идея ResNet заключается во введении *skip connections*, которые позволяют сигналу обходить один или несколько слоёв сети. Это означает, что каждый последующий слой учится не непосредственно преобразовывать входные данные, а корректировать или добавлять остаток к выходу предыдущего слоя. Такой подход предотвращает проблемы с затуханием или взрывом градиентов.

Учитывая успешное применение ResNet в компьютерном зрении [10] и его недавние достижения в обработке естественного языка NLP [17], я решил использовать модифицированную архитектуру ResNet в задаче кредитного скоринга. Структурный блок упрощён по сравнению с оригинальной архитектурой: фактически это MLP с добавлением батч нормализации и skip connection.



2.2 Трансформеры

2.2.1 Что такое трансформер?

Как известно, почти все значимые достижения последних лет в области машинного обучения так или иначе связаны с трансформерами. Эта простая с точки зрения идеи, но невероятно эффективная архитектура произвела настоящий фурор в области глубинного обучения. Впервые эта архитектура была предложена группой исследователей Google в работе *“Attention is All You Need”, Vaswani [9]*.

Ключевая идея трансформера заключается в добавлении специального механизма самовнимания (self-attention), благодаря которому модель способна улавливать взаимосвязи не только между соседними входными элементами, но и между далекими токенами. Такой механизм дает модели возможность находить зависимости между элементами, даже если они находятся на расстоянии сотен и тысяч токенов, что позволило обрабатывать большие тексты. Это привело к прорывам в машинном переводе, суммаризации и генерации текстов, а также во многих других областях. Например: различные модификации трансформеров стали основой для моделей *“GPT”, Radford, 2018 [19]* и *“BERT”, Devlin, 2018 [20]*

После успеха в NLP, трансформеры нашли применение и в других сферах. Например, не так давно в компьютерном зрении были разработаны модели, такие как Vision Transformer [18], которые показали конкурентоспособные результаты в сравнении с традиционными сверточными нейронными сетями. Трансформеры также применяются в обработке временных рядов, речи, геномных данных и даже в генеративных задачах, демонстрируя универсальность и мощь механизма внимания.

2.2.2 Трансформер в кредитном скоринге - TabTransformer

Учитывая впечатляющие успехи трансформерной архитектуры описанные выше, закономерно встал вопрос о применении ее для работы с табличными данными.

Первой значимой работой в этой сфере стала статья “*TabTransformer*“, *Xin Huang* [24], где была предложена идея передачи категориальных фичей в трансформер через стандартные контекстные эмбединги. Далее выход трансформера (а именно <CLS> токен) конкатенируется с численными признаками и подается в MLP.

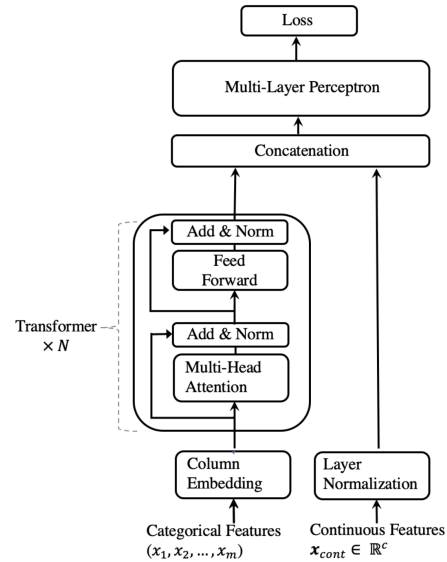


Рис. 1: Архитектура TabTransformer

Основной проблемой в архитектуре TabTransformer является не самая оптимальная обработка численных признаков. Фактически для численных признаков эта модель сопоставима обычному MLP, поэтому она давала лишь небольшой прирост метрик относительно полносвязной нейронной сети.

Идея данного исследования состоит в том, чтобы предложить архитектуру, которая будет более эффективно обрабатывать численные признаки.

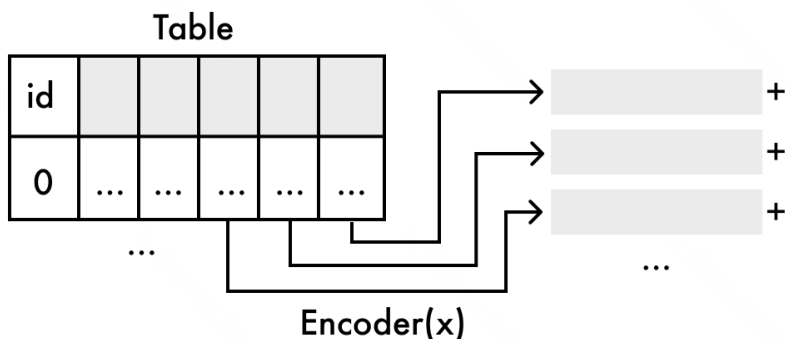
2.2.3 FT-Transformer

Дальнейшее развитие этой идеи не заставило себя долго ждать: уже через год, исследователи из Yandex Research предложили усовершенствовать модель и подать в трансформер все признаки, а не только категориальные. Эта модель была представлена в статье *“Revisiting Deep Learning Models for Tabular Data”*, Gorishniy [1] и получила название FT-Transformer (Feature Tokenizer + Transformer). Фактически,

она ничем не отличается от стандартного TabTransformer, за исключением того, что численные признаки также через эмбединги подаются на вход в трансформерные слои.

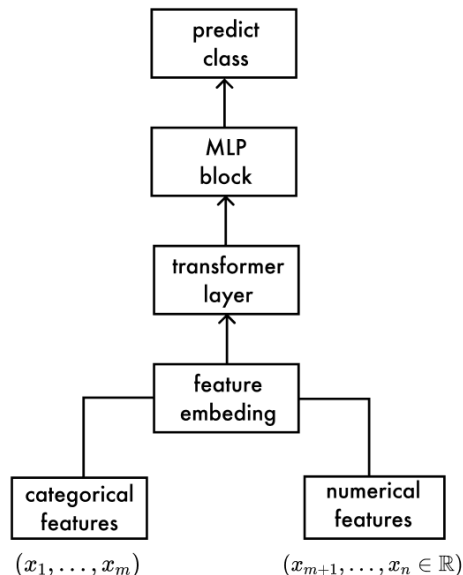
В рамках данного исследования также была протестирована архитектура, построенная на базе трансформера, которая является модернизацией FT-Transformer. Как видно из схемы выше, архитектура данной нейронной сети состоит из нескольких ключевых частей:

1. Энкодер - преобразует каждый признак в таблице в токен (вектор из чисел). Способ преобразования значения признака в токен отличается для категориальных и численных признаков (это обсудим позже, в пункте про эмбединги):

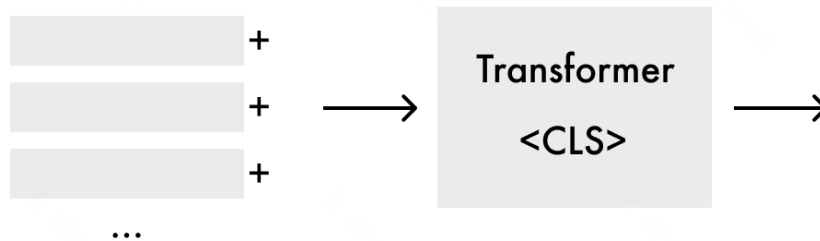


2. После предыдущего слоя формируется последовательность токенов для каждого объекта, то есть для каждой строки таблицы. Эти токены подаются в слои

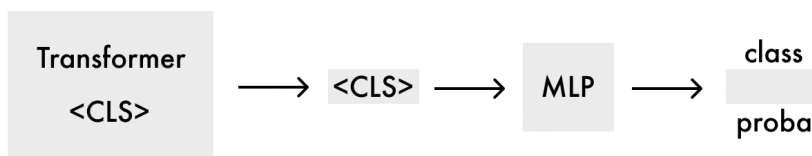
FT-Transformer



трансформера как последовательные данные, то есть трансформер работает с последовательностью токенов, которые в действительности являются характеристиками одного объекта:



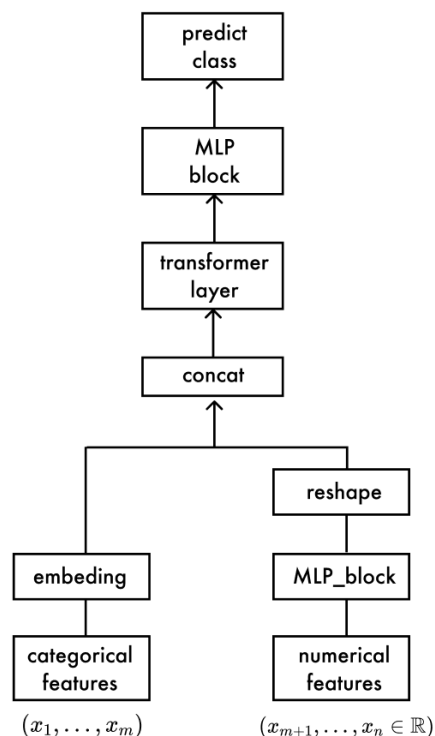
- После этого используется <CLS> токен (специальный токен, который добавляется в начало входной последовательности и используется для агрегации информации всего входного контекста), который подается на вход в обычную полносвязную нейронную сеть и на выходе получается предсказание класса:



2.2.4 MLP encoder Transformer

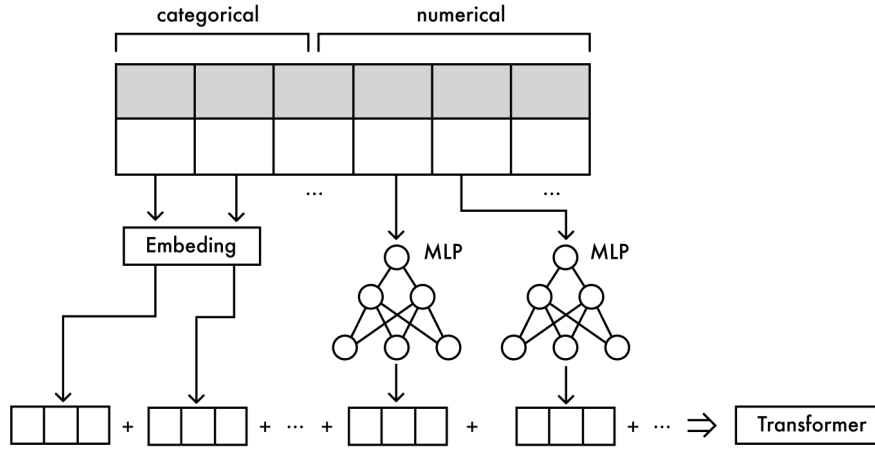
Развивая эту идею далее, можно рассмотреть еще одну интересную архитектуру. Как упоминалось ранее, основная проблема TabTransformer - неоптимальная обработка численных признаков. Модель из предыдущего пункта решала эту проблему путем перевода численных признаков в специальные токены, которые, как будет показано далее, не обучаемые и зависят только от значения конкретного признака и его распределения.

MLP_encoder_Transformer



Идея **MLP encoder Transformer** – сделать эти эмбединги обучаемыми (аналогично тому, как это работает в контекстных эмбедингах в NLP).

Рассмотрим как это будет работать: каждый численный признак подается в блок MLP для того чтобы из скалярных величин получить векторные представления - токены, размерность которых будет совпадать с размерностью эмбедингов категориальных признаков:



В остальном, принцип работы модели не меняется: полученная последовательность токенов передается в трансформерные слои, аналогично предыдущей архитектуре. На выходе получается <CLS> токен, который далее преобразуется в предсказание класса.

2.3 Примеры эмбедингов

Эмбединг - очень важный слой нейронной сети, позволяющий преобразовывать категориальные и численные признаки в векторные представления - токены. Эмбединг для значения признака x_i вычисляется следующим образом:

$$T_i = f_i(x_i) \in \mathbb{R}^d \quad f_i : \mathbb{X}_i \rightarrow \mathbb{R}^d, \quad (4)$$

В современном глубоком обучении эмбединги играют ключевую роль в подготовке данных ко входу в модель. Недавние исследования [2] подчеркивают важность эмбедингов в повышении эффективности трансформерных архитектур нейронных сетей. Правильный выбор эмбедингов может существенно повлиять на качество модели,

в связи с этим, в данном исследовании будет рассмотрено несколько различных типов эмбедингов, чтобы определить их влияние на эффективность модели в задаче кредитного скоринга.

2.3.1 Element-wise multiplication encoding

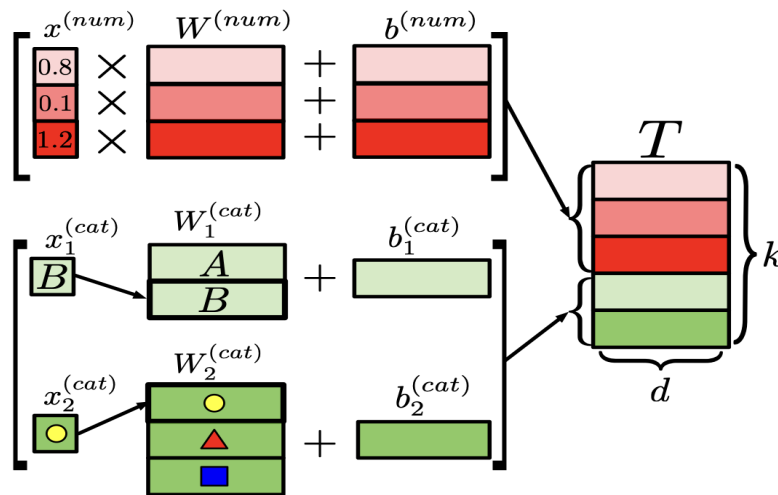
Самый наивный вариант эмбединга, предложенный в работе о применении нейронных сетей на табличных данных от *Yandex Research* [1]. Принцип вычисления i -го токена основан на поэлементном произведении обучаемого вектора весов W_i на значение признака x_i . Для категориальных признаков во всех архитектурах применяется эмбединг, основанный по принципу словаря с обучаемыми весами (CategoricalEncoder):

$$T_i^{(\text{num})} = b_i^{(\text{num})} + x_i^{(\text{num})} \cdot W_i^{(\text{num})} \in \mathbb{R}^d, \quad (5)$$

$$T_i^{(\text{cat})} = b_i^{(\text{cat})} + e_i^T W_i^{(\text{cat})} \in \mathbb{R}^d, \text{ где } e_i^T \text{ — one-hot вектор для } x_i, \quad (6)$$

$$T = \text{concat} \left[T_{1^{(\text{num})}}^{(\text{num})}, \dots, T_{k^{(\text{num})}}^{(\text{num})}, T_{1^{(\text{cat})}}^{(\text{cat})}, \dots, T_{k^{(\text{cat})}}^{(\text{cat})} \right] \in \mathbb{R}^{k \times d}. \quad (7)$$

Пример эмбединга для таблицы с тремя численными и двумя категориальными фичами:



2.3.2 Piecewise linear encoding

Несколько новых эмбедингов для численных признаков на табличных данных были предложены в статье “On Embeddings for Numerical Features in Tabular Deep

Learning”, Yury Gorishniy [2]. Один из них представляет собой усовершенствованный подход бинирования (разбиение на бины).

Идея этого эмбединга очень проста и понятна: метод вдохновлен самым популярным в классическом машинном обучении кодированием - **One-Hot-Encoding**, который широко и успешно используется для представления категориальных признаков.

Фактически, это непрерывная альтернатива **one-hot** кодированию: для i -го численного признака его диапазон значений разбивается на множество из T^i непересекающихся интервалов $B_1^i, \dots, B_T^i$ - бинов:

$$\text{PLE}(x) = [e_1, \dots, e_T] \in \mathbb{R}^T \quad (8)$$

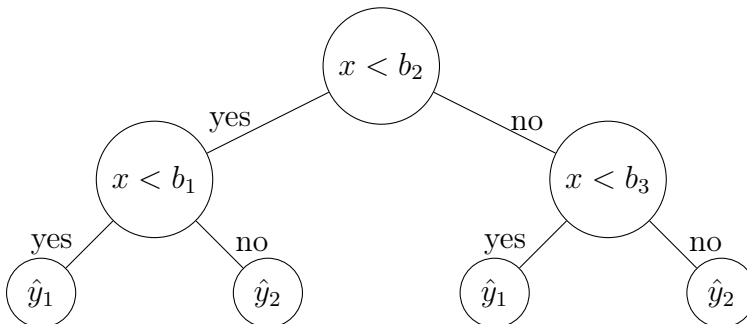
$$e_t = \begin{cases} 0, & x < b_{t-1} \text{ и } t > 1 \\ 1, & x \geq b_t \text{ и } t < T \\ \frac{x-b_{t-1}}{b_t-b_{t-1}}, & \text{иначе} \end{cases} \quad (9)$$

Способов выбора бинов $B_1^i, \dots, B_T^i$ может быть несколько. Вот некоторые интересные из них:

1. Разделение диапазона значений в соответствии с равномерно выбранными квантилями распределения соответствующего признака на обучающей выборке:

$$b_t = Q_t \left(\left\{ x_i^{(j)} \right\}_{j \in J_{train}} \right), \text{ где } Q \text{ — это функция получения квантиля.} \quad (10)$$

2. Построение решающего дерева по одному признаку в таргет. Принципиальное отличие этого подхода заключается в том, что в нем используются обучающие метки классов для построения бинов:



2.3.3 Periodic encoding

В последние 5 лет периодические функции стали ключевым компонентом для обработки (эмбедингов) численных входных данных. Примерами применения такого рода токенизаторов являются обработка естественного языка [9], компьютерное зрение [21], неявные нейронные представления [22]. В отличие от бинирования, данный метод сохраняет информацию о количественных различиях между признаками:

$$f_i(x) = \text{Periodic}(x) = \text{concat}[\sin(v), \cos(v)] \quad \forall v = [2\pi c_1 x, \dots, 2\pi c_k x] \quad (11)$$

В данной работе также исследуется эффективность применения фурье-эмбедингов.

3 Данные и их обработка

3.1 Данные

Для экспериментов были выбраны публичные наборы данных, предоставленные крупными банками (например *Home Credit Bank*, *Raiffeisen*) для соревнований в области оптимизации эффективности кредитного скоринга на платформе **Kaggle**:

1. **HCDR** - датасет с соревнования Home Credit Default Risk², 2018. Содержит порядка 300 тысяч кредитных заявок клиентов с более чем 120-ю признаками из семи различных источников: выписки кредитного бюро, банковские транзакции, кредитная история, скоры внешних источников и др. Целевая переменная — факт дефолта по кредиту.
2. **CSC** - стандартный тренировочный датасет задачи кредитного скоринга Credit score classification³, 2024. Он содержит порядка 50-и тысяч кредитных заявок клиентов с более чем 30-ю признаками, таргет - факт дефолта по кредиту.
3. **ADP** - датасет с соревнования American Express - Default Prediction⁴, 2021. Задача соревнования - предсказать вероятность того, что клиент не погасит задолженность по его кредитной карте, в будущем, основываясь на его ежемесячном профиле. Таргет - отсутствие просрочки на протяжении 180 дней после открытия карты.

Во всех датасетах доля таргетов порядка 4 — 8%, что сразу же указывает на сильный классовый дисбаланс

3.2 Обработка данных

Для обработки данных был реализован специальный класс `PrepareTabularData`, который выполняет все этапы предварительной обработки данных для каждой из исследуемых архитектур нейронной сети - по разному. Порядок работы класса следующий:

²<https://www.kaggle.com/competitions/home-credit-default-risk>

³<https://www.kaggle.com/datasets/parisrohan/credit-score-classification?select=train.csv>

⁴<https://www.kaggle.com/competitions/amex-default-prediction/data>

1. Загружает данные, разделяет все признаки на категориальные и численные (определение типа происходит по количеству уникальных значений), заполняет пропуски, для удобства дальнейшей обработки признаков делит датасет на три части - категориальные признаки, численные признаки и таргет.
2. Разделяет данные на `train : test : validate` в пропорциях $1 - \alpha - \beta : \alpha : \beta$. При разделении данных используется стратификация, чтобы сохранить исходное соотношение классов.
3. Обработывает признаки:
 - **Численные признаки** - нормализует (по умолчанию при помощи `StandardScaler`) и токенизирует их при помощи одного из исследуемых энкодеров (по умолчанию применяется `PiecewiseLinearEncoder`).
 - **Категориальные признаки** - применяет `OneHotEncoder` в случае бустинга или `OrdinalEncoder` для нейронных сетей, чтобы потом их можно было подать в `CategoricalEncoder`.
4. Преобразует полученные данные в `PyTorch`-тензоры и переносит их на `GPU`, если это возможно.

4 Эксперименты

4.1 Метрики качества модели

В задаче кредитного скоринга, как упоминалось ранее, мы сталкиваемся со стандартной для задач классификации проблемой - сильным дисбалансом классов. Это в первую очередь связано со спецификой задачи кредитного скоринга - банку просто невыгодно выдавать кредиты людям, которые деньги не вернут, поэтому таргетов (выходов в дефолт) в выборке обычно очень мало - порядка 2 – 5%. В подобных ситуациях классическая метрика классификации - точность предсказания (*accuracy*) становится малоинформативной, потому что даже модель предсказывающая всегда класс 0 будет иметь высокие показатели ($accuracy = 0.98$), но, очевидно, окажется абсолютно бесполезной для выявления рисков, поэтому, для корректной оценки качества модели, будут использоваться метрики устойчивые к дисбалансу.

4.1.1 Precision и Recall

Для подсчета точности и полноты используются общепринятые обозначения - TP (истинно положительные), TN (истинно отрицательные), FP (ложно положительные) и FN (ложно отрицательные) значения:

1. $Precision = \frac{TP}{TP+FP}$ - полнота (доля верных предсказаний дефолтов)

2. $Recall = \frac{TP}{TP+FN}$ - точность (покрытие реальных дефолтов)

4.1.2 ROC-AUC

ROC-AUC - устойчивая к дисбалансу метрика, оценивающая способность модели ранжировать объекты по вероятности принадлежности к конкретному классу (в нашем случае - вероятности дефолта клиента). ROC-AUC - площадь под ROC-кривой - это кривая, построенная по принципу: по оси x - полнота, а по оси y - точность. То есть:

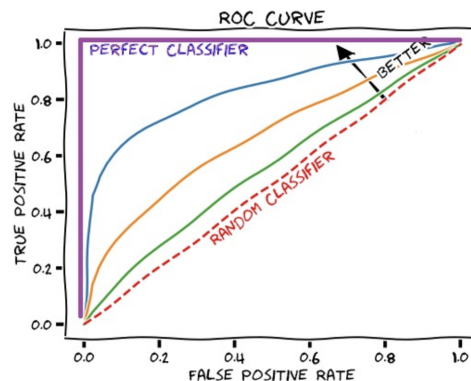


Рис. 2: ROC-кривая

$$\text{ROC-AUC} = \int_0^1 \text{TPR}(FPR)d(FPR)$$

- обычно такой интеграл подсчитывается при помощи метода трапеций.

Полученное в эксперименте значение ROC-AUC можно проинтерпретировать так:

- 1 - идеальная модель
- 0.5 - абсолютно случайное угадывание класса
- 0 - идеальная модель, предсказывающая классы в точности наоборот (перепутаны таргеты)

P.S. Многие крупные банки зачастую вместо ROC-AUC используют коэффициент Джини - его отнормированную версию: $\text{GINI} = 2 \cdot \text{AUC} - 1 \in [-1, 1]$

4.1.3 Выводы о метриках

Перечисленные выше метрики в совокупности позволяют оценить модель с разных точек зрения: ROC-AUC характеризуют способность модели к ранжированию объектов, что на самом деле является ключевым качеством скоринговой модели, Precision и Recall показывают эффективность работы с классом дефолтов - на практике для скоринга эти метрики не используются, но зато применяются в задаче кредитного и транзакционного антифрода.

Разумеется, в реальной жизни все модели кредитного скоринга оцениваются по бизнес метрикам: например, по сумме просрочки (exposure at default), сумме выданных кредитов (GMV), NPV (Net Present Value) и так далее.

Но, поскольку доступа к таким данным нету, валидация моделей производилась по техническим метрикам, о которых было сказано выше, и которые являются хорошими прокси-метриками для бизнесовых показателей.

4.2 Эксперименты

Эксперименты для всех архитектур и эмбедингов проводились в облачной платформе *YandexCloud* на графической карте **NVIDIA A100** или **NVIDIA V100**. Обучение каждой модели нейронной сети (кроме **MLP encoder Transformer**, которая из-за случайной инициализации эмбедингов обучается сильно дольше - порядка 60-и эпох) проводилось в течение примерно 30-и эпох, чтобы избежать переобучения, но в то же время получить хорошее качество. Использовался стандартный оптимизатор **Adam** и лосс-функция **BinaryCrossEntropy** (бинарную кросс-энтропию). Для проверки качества моделей была выбрана метрика *ROC-AUC*, как самая эффективная в задаче бинарной классификации с дисбалансом данных. Кроме того, эта метрика является самой популярной для кредитного скоринга в большом бизнесе.

4.2.1 Структура репозитория

Все предложенные выше модели были реализованы на языке **Python** с использованием библиотеки **PyTorch**, исходный код лежит в репозитории **GitVerse**⁵. Весь код лежит в папке `experiments/src/`, структура которой устроена следующим образом:

- `num_encoders/` - все реализованные энкодеры для численных признаков
- `cat_encoders/` - энкодер для категориальных признаков
- `nn_models/` - классы всех предложенных архитектур нейронных сетей
- `trainer` - класс для обучения любой из предложенных моделей
- `prepare_dataset` - класс для обработки датасета и подготовки его для подачи в модель

4.2.2 Сравнение бейзлайнов моделей

Прежде всего, посмотрим на эффективность описанных во второй главе моделей нейронных сетей (**MLP**, **ResNet** и **FT-Transformer**) без применения специфичных эмбедингов, то есть при подаче данные напрямую в модель. Полученные результаты

⁵[Tabular_neural_networks](#)

сравнивались с одной из самых популярных и эффективных реализаций градиентного бустинга из библиотеки CatBoost.

Таблица 1: ROC-AUC на валидационном датасете

Model	HCDR	ADP	CSC
MLP	0.738	0.731	0.925
ResNet	0.723	0.703	0.915
FT-Transformer	0.748	0.742	0.926
CatBoost	0.761 ↗	0.750 ↗	0.939 ↗

Результаты в таблице 1 демонстрируют уверенное преимущество градиентного бустинга на всех датасетах, причем с большим отрывом в ~ 1 процентный пункт. Следовательно простое применение стандартных архитектур не позволяет получить результаты сопоставимые с бустингом - нужно модернизировать архитектуры для работы с табличными данными.

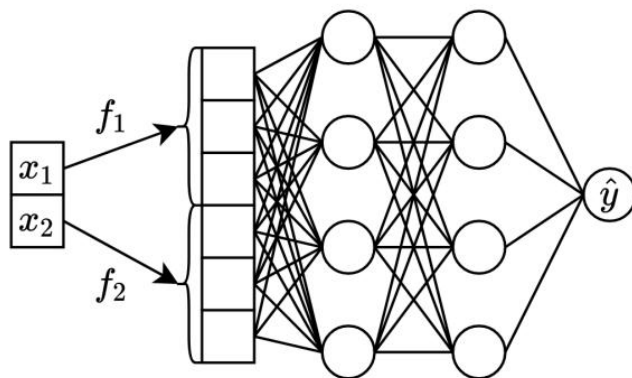
4.2.3 Влияние эмбедингов численных признаков на качество модели

Далее, с целью улучшения качества моделей были исследованы различные виды токенизации численных признаков. На примере базовой архитектуры MLP были протестированы три типа эмбедингов и вариант без какой-либо специальной обработки:

- WithoutEncoder – без необычного кодирования: численные признаки подаются в исходном виде
- PeriodicEncoder – фурье-кодирование, преобразующее численные признаки в пространство синусов и косинусов
- PiecewiseLinearEncoder – кусочно-линейное преобразование
- ElementWiseMultiplEncoder – поэлементное умножение на матрицу весов

В данной архитектуре эмбединги применяются так: каждый признак после прохождения слоя эмбединга разворачивается в токен (вектор чисел), после этого все полученные векторы конкатенируются и подаются на вход полносвязного слоя. Как можно заметить, размерность входа в MLP в таком случае увеличивается в

`embedding_size` раз, значит сильно растет и количество обучаемых параметров. Архитектура в данном случае выглядит так:



В результате эксперимента, были получены такие результаты:

Таблица 2: Влияние различных энкодеров на качество MLP модели

Энкодер	HCDR	ADP	CSC
Without Encoder	0.738	0.744	0.931
PeriodicEncoder	0.712	0.683	0.932 ↗
PiecewiseLinearEncoder	0.752 ↗	0.753 ↗	0.917
ElementWiseMultiplEncoder	0.72	0.712	0.896

В среднем наилучшие результаты показывает кусочно-линейное кодирование, поэтому в итоговом эксперименте для сравнения моделей оно будет использоваться как основное для всех архитектур.

4.2.4 Финальный эксперимент

В финальном эксперименте были рассмотрены бейзлайновые модели - MLP, ResNet с применением кусочно-линейного эмбединга и модели основанные на механизме внимания - PLE-encoder Transformer (также вариант с PLE-кодированием), MLP-encoder Transformer (обучаемые эмбединги численных признаков). Полученные результаты (смотреть таблицу 3) демонстрируют высокую конкурентоспособность нейронных сетей, основанных на идее трансформера и адаптированных под табличные данные. Да, возможно, на данный момент бустинг все же остается предпочтительным способом из-за своей простоты и эффективности, но потенциал развития трансформеров в задаче кредитного скоринга - огромен.

Таблица 3: *ROC-AUC* на валидационном датасете

Model	HCDR	ADP	CSC
PLE-encoder MLP	0.753	0.745	0.925
PLE-encoder ResNet	0.727	0.729	0.920
PLE-encoder Transformer	0.758	0.756 ↗	0.943 ↗
MLP-encoder Transformer	0.733	0.743	0.928
CatBoost	0.761 ↗	0.750	0.939

Также хотелось бы отметить, что использование механизма внимания дает значительный прирост качества по сравнению с бейзлайновыми архитектурами MLP, ResNet, за счет чего трансформерные модели стабильно превосходят их на всех трех датасетах.

Кроме того, архитектура MLP-encoder Transformer обучается гораздо дольше остальных из-за случайной стартовой инициализации эмбедингов. Здесь возможно дальнейшее развитие идеи: например, можно инициализировать эмбединги кусочно-линейным способом, а потом тюнить их в процессе обучения. Такой подход может привести к росту эффективности модели.

5 Заключение

В ходе данной работы были предложены несколько архитектур нейронных сетей, основанных на механизме внимания, а именно - различных адаптаций модели трансформера. Ключевой особенностью данных архитектур является внедрение в них специальных методов векторизации численных признаков (эмбедингов), поскольку первоначальные эксперименты проведенные с бейзлайнами моделей не привели к успеху.

Исследование влияния типа эмбедингов численных признаков выявило, что подбор правильной токенизации является критически важным для качества моделей. В частности, кусочно-линейное кодирование (**PLE Encoder**) в среднем показало наилучший прирост метрик.

Все модели сравнивались на нескольких публичных датасетах с классическим градиентным бустингом, а именно с одной из самых популярных его реализаций – **CatBoost**-ом.

Финальные эксперименты показали, что предложенные трансформерные модели с применением эффективных эмбедингов вполне могут конкурировать с градиентным бустингом и показывать сопоставимые по качеству результаты, а иногда даже превосходить его по эффективности.

Таким образом, данное исследование опровергает распространенный тезис об абсолютном доминировании бустинга в задаче кредитного скоринга. Оно наглядно демонстрирует огромный потенциал нейронных сетей, ведь при грамотном проектировании и настройке трансформеры уже сейчас являются высокоэффективной альтернативой этому методу.

Кроме того, стоит отметить, что потенциал предложенных идей не до конца исчерпан. Например, в ходе работы было уделено мало времени оптимизации гиперпараметров, что открывает возможности для дальнейших исследований. Особенно интересной идеей является инициализация энкодера в модели **MLP encoder Transformer** не случайным вектором, а, например, весами предварительно настроенного кусочно-линейного энкодера.

Список литературы

1. Yury Gorishniy, Ivan Rubachev, Valentin Khrulkov, Artem Babenko - Revisiting Deep Learning Models for Tabular Data, arXiv, 2106.11959, 2022
2. Yury Gorishniy, Ivan Rubachev, Artem Babenko - On Embeddings for Numerical Features in Tabular Deep Learning, arXiv, 2203.05556v4, 2023
3. Noam Shazeer - GLU Variants Improve Transformer, arXiv, 2002.05202, 2020
4. P Aditya Sreekar, Sahil Verma, Varun Madhavan, Abhishek Persad - Unveiling the Power of Self-Attention for Shipping Cost Prediction: The Rate Card Transformer, arXiv, 2311.11694, 2023
5. Yury Gorishniy, Akim Kotelnikov, Artem Babenko - TABM: Advancing Tabular Deep Learning With Parameter-efficient Ensembling, arXiv, 2410.24210, 2024
6. Vadim Borisov, Tobias Leemann, Kathrin Seßler, Johannes Haug, Martin Pawelczyk and Gjergji Kasneci - Deep Neural Networks and Tabular Data: A Survey, arXiv, 2110.01889, 2024
7. Наим Сиддики - Скоринговые карты для оценки кредитного риска,
8. Günter Klambauer, Thomas Unterthiner, Andreas Mayr - Self-Normalizing Neural Networks, arXiv, 1706.02515, 2017
9. Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin - Attention Is All You Need, arXiv, 1706.03762, 2017
10. Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun - Deep Residual Learning for Image Recognition, arXiv, 1512.03385, 2015
11. Irwan Bello, William Fedus, Xianzhi Du, Ekin D. Cubuk, Aravind Srinivas, Tsung-Yi Lin, Jonathon Shlens, Barret Zoph - Revisiting ResNets: Improved Training and Scaling Strategies, arXiv, 2103.07579, 2021

12. Ira Shavitt, Eran Segal - Regularization Learning Networks: Deep Learning for Tabular Datasets, arXiv, 1805.06440, 2018
13. Sercan O.Arik, Tomas Pfister - TabNet: Attentive Interpretable Tabular Learning, arXiv, 1908.07442, 2019 (Tab Net)
14. Sarkhan Badirli, Xuanqing Liu, Zhengming Xing, Avradeep Bhowmik, Khoa Doan, Sathiya S. Keerthi - Gradient Boosting Neural Networks: GrowNet, arXiv, 2002.07971, 2020 (Бустинг Нейросеток)
15. Sergei Popov, Stanislav Morozov, Artem Babenko - Neural Oblivious Decision Ensembles for deep learning on tabular data, arXiv, 1909.06312, 2019 (Имитация деревьев нейросетями)
16. Gowthami Somepalli, Avi Schwarzschild, Micah Goldblum, C. Bayan Bruss, C. Bayan Bruss, Tom Goldstein - SAINT: Improved Neural Networks for Tabular Data via Row Attention and Contrastive Pre-Training, arXiv, 2106.01342, 2021 (Трансформеры на табличных данных)
17. S. Sun, M. Iyyer - Revisiting simple neural probabilistic language models, arXiv, 2104.03474v1, 2021
18. Qiming Zhang, Jing Zhang, Yufei Xu, Dacheng Tao - Vision Transformer with Quadrangle Attention, arXiv, 2303.15105, 2023
19. Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever - Improving Language Understanding by Generative Pre-Training, 2018
20. Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova - BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, arXiv, 1810.04805, 2018
21. Y. Li, S. Si, G. Li, C. Hsieh, and S. Bengio - Learnable fourier features for multi-dimensional spatial positional encoding, arXiv, 2106.02795, NeurIPS, 2021
22. B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, R. Ng. Nerf - Representing scenes as neural radiance fields for view synthesis, arxiv, 2003.08934, ECCV, 2020

23. Gowthami Somepalli, Micah Goldblum, Avi Schwarzschild, C. Bayan Bruss, Tom Goldstein - SAINT: Improved Neural Networks for Tabular Data via Row Attention and Contrastive Pre-Training, 2106.01342, 2021
24. Xin Huang, Ashish Khetan, Milan Cvitkovic, Zohar Karnin - TabTransformer: Tabular Data Modeling Using Contextual Embeddings, arXiv, 2012.06678v1, 2020
25. Scott M. Lundberg, Su-In Lee - A Unified Approach to Interpreting Model Predictions (SHAP), arXiv, 1705.07874, 2017